

Date: / /

oOP

Topics :

- class / objects
- Attributes / Methods
- object lifecycle
- constructors (Default , Parameterized)
- Access Modifiers (Public , Private , Protected)
- The 4 Pillars of oop
 - Encapsulation
 - Getters , setters
 - Inheritance
 - Single , Multiple , Multilevel
 - Polymorphism
 - Method overloading , Method overriding
 - Abstraction
 - Abstract class , Interfaces
- Some Keywords

Date: / /

Destructor

← الجافا يعتمد كلياً على الـ "Garbage collector" ويتكفل بنفسه بمسح الذاكرة من مستوى الـ (Memory) ومن هنا تظهر المشكلة لايتكفل في object له علاقة بالـ External Resources (Database connections) لأنه لايعرف علاقة بين الـ (I/O system) والـ Database connections

عشانه كده بنلجا لاستخدام حاجات نزي
try-with-resources ← أو الالة close()

لمنع مشاكل نزي الـ Recourse leak

Access Modifiers

- عبارة عن Keywords مخصصة في اللغة بيتم استخدامها لكتابة صيغ هيستخدام لايه وظيفه

1. Public ← أي مكانه في البرنامج بيكون لي Access عليها سواء كانت في نفس الـ Package أو في غيرها

2. Private ← أي كلاس نفسه هو الوحيد اللي يقدر Access عليه

3. Protected ← أي كلاس نفسه + أي كلاس معاه في نفس الـ Package + أي كلاس وارث منه

4. default ← Package Private يعني أي كلاس في نفس الـ Package

الفرق بين الـ "default" و "Protected":

الـ Protected يسمح للـ sub classes بالوصول على لوكانوا في Package مختلفه

Date: / /

The Pillars of oop

1.

1. Encapsulation

* مبدأ التغليف يعتمد على مرحلتين
Access Modifiers ←
Control Access (getters, setters)

* غالباً يتم تحويل جميع الـ "Attributes" إلى Private لمنع الوصول المباشر
وذلك لسبب الأمان منع الـ Access من غير منقضية الـ
Attributes عن طريق إضافة الـ Access والشروط في الـ setters

* التغليف من مبررات إخفاء البيانات ولكنه هو تجميع البيانات والموارد في مكان واحد

2. Inheritance

الاستفادة (إكسبري) هي "Reusability", "عدم التكرار" وبالتالي توفير وقت ومجهود

* الموضوع يتم عن طريق إنشاء الـ shared component التي هي أكثر من class ونحطها في الـ super class ولديها كلاس مشترك مع الـ shared component
يؤثر منه عن طريق الـ keyword extends

* في لوحيك ننشئ من الـ class يكون مدمج فيه الخصائص الموجودة في الـ class
وذلك يتم عن طريق إنشاء الـ constructor مدمج الـ class يكون فيها calling
الـ default constructor مدمج الـ class مدمجاً وذلك لإنشاء نقطة مدمجة
وهو الذي لا نحتاج إنشاء الـ default constructor مدمج الـ class
لأنه لو عايناهم نرى calling الـ constructor مدمج في الـ class
فالموضوع هو يتم عن طريق الـ super keyword ونوع وعدد الـ parameters
التي يدمجها

Date: / /

super keyword لا يتم استخدامه في أول سطر
super يشاور على ال components من ال class
أو في حالة إنشاء أو كائن من ال class
جاءا يمنع ال Multiple Inheritance على مستوى ال classes
ويمنع ال Multi level Inheritance

3. Polymorphism

Method overloading ← اتفاق لأكثر من دالة في الاسم واختلاف في
Parameters في نوع وعدد ال compile-Time

يتم ما يلي لأكثر من دالة في نفس الكلاس

Method overriding ← يتم ما يلي دالة واحدة في كلاس الأب وواحدة في الكلاس
Run-Time
عن الموضع مرتبط بالوراثة بشكل أساسي

← ال signature يتفق في ال
Implementation ويتفق في ال

← ال static methods ممنوع تعدي ال override

Date: / /

4- Abstraction.

- 1- ال Abstract class ممنوع تنشئ منه كائنات بطريقة مباشرة
- 2- " " " " يتم توريثه
- 3- يحتوي على Abstract method و Normal method ^{التي يمكن استخدامها} _{بشكل مباشر}
- 4- يمكن نقل Multilevel inheritance في ال Abstract classes
- 5- يمكن الدبغ التي هي وراثت Abstract class لا تزم بعمل override
- 6- كل ال Methods الموجودة في الآباء كلهم
- 7- يمكن محتوى متغيرات من أي نوع Protected, Private, Public
- 8- يمكن محتوى constructor عامي جداً

Interfaces ← تجريد 100%

- 1- الكلاس يقدر ينفذ أكثر من Interface في نفس الوقت
- 2- كل المتغيرات بتكونه Public static final ← by default
- 3- معناه هوش Constructor نهائياً
- 4- بعد Java 8 يقدر محتوى على دوال ليس Implementation بشرط بتكونه ^{default} أو _{static}

static, final

Class ← ممنوع توريث
Variable ← قيمته ثابتة ممنوع التحويل عليها } final
Method ← ممنوع بتغييره overriding }

static ← لأي حاجة static بتكون خاصة بالكلاس نفسه
من أي كائنات منه

Date: / /

قواعد الـ Java

class to class → extends

Interface to Interface → extends

لا يمكن أن يمتد
Interface إلى Interface

class to Interface → implements

Interface to class → لا يمكن أن يمتد